

Technical Assessment: Retail Pricing Intelligence Platform

Objective

Build an early version of a retail pricing intelligence system that helps merchandisers understand how their products are priced relative to competitors and identify potential repricing opportunities efficiently.

This assessment mirrors real-world engineering: you'll have incomplete requirements, messy data, time constraints, and need to make judgment calls.

Time Expectation & Use of AI Tools

- Expected time commitment: ~4–8 hours
 - Focus on building something thoughtful and demonstrable
 - Production-level completeness is not expected
 - Use of AI assistance is encouraged
 - Feel free to use AI tools (e.g. ChatGPT, Copilot, Cursor, LLMs, Figma AI) for design, coding, debugging, analysis, or documentation
 - If helpful, briefly note how AI tools supported your work in DECISIONS.md
-

Context

You're the first engineering hire at a retail analytics startup.

The merchandising team regularly evaluates competitor pricing to decide whether products should be repriced, monitored, or left unchanged. They currently rely on spreadsheets and manual analysis. The business has scraped an initial dataset and wants to move toward a proper system that supports faster, more consistent decisions.

You are given a CSV file (`pricing_data.csv`, ~5,000 rows) containing internal product data, competitor listings, and match-quality metrics. **The data is messy**—treat it like real-world input.

What the System Should Enable

Your solution should consider what would actually be useful for merchandisers

- Can they see which products are overpriced at a glance?

- Can they identify patterns (e.g., is a specific competitor always cheaper)?
- Can they find actionable items quickly (e.g. products that require repricing attention)?
- Are there observable pricing patterns across categories or vendors?

How these questions are answered—through tables and summary statistics, visualizations with filters, conversational chat or other interfaces—is up to you.

Some indicators you *might* consider (non-exhaustive):

- Price index (our price ÷ competitor price)
- Overpricing flags (e.g. >10% above competitors)

We're more interested in your reasoning than checking boxes.

Scope & Requirements

Must-Haves

- Use a real database (your choice—but tell us why)
- Build a web-based interface that buyers can use
 - Framework/approach is your choice
 - Should display pricing data in some useful way
- Handle the data quality issues in the CSV
- Make it runnable locally with clear setup instructions

Nice-to-Have (Optional)

- A conversational or query-style interface (e.g. natural language questions)
- If included, it should complement—not replace—the main interface

You Decide. Part of this exercise is seeing what *you* think matters.

- What specific features to build
 - How to present the data to buyers
 - How to structure the data model
 - What to build now vs. document for later
-

Deliverables

1. Working Code

- GitHub repository (private with access granted)
- Clear instructions to run it locally

2. DECISIONS.md (Required)

This is where you demonstrate how you think. Walk us through your decision-making process:

- **Backend Architecture**
 - Data storage choices
 - Schema and aggregation strategy
 - Alternatives considered
- **Scope & Prioritization**
 - What you built and why
 - What you intentionally left out
- **Data Quality**
 - Issues discovered and how you handled them
 - What you'd improve with more context
- **User Experience**
 - Who the system is for
 - How your interface supports their workflow
- **Looking Ahead**
 - Known limitations
 - What you'd build next

3. README.md

- Setup steps
 - Dependencies
 - Known issues or constraints
-

Evaluation Criteria

We're looking for evidence that you can:

- Make pragmatic technical decisions under constraints
- Think about the actual users and their needs
- Handle messy, real-world data
- Write clean, understandable code
- Communicate decisions effectively

We are not evaluating polish, scale, or feature completeness.

Submission

Send us:

1. Link to GitHub repository (or zip file if you prefer). Please ensure that your GitHub repository is private and shared with us only.
2. Any additional context you think is relevant

We will review your submission and get back to you within 1 week.

Good luck! We're excited to see what you build.

Appendix: Data Schema Definition

1. Reference Product (MR.DIY)

Fields identifying the internal product.

Field Name	Type	Description
product_id_diy	String	Primary Key. Unique identifier for the MR.DIY product.
s_title_diy	String	Product name.
s_price_diy	Float	Price in local currency (MYR).
s_price_value_diy	Float	Original price value before standardization.
s_price_per_unit_diy	Float	Price per standardized unit (e.g., per ml, per gram).
s_primary_image_diy	URL	Link to the product image.
s_link_diy	URL	Link to the product page.
s_brand_diy	String	Brand name (if available).
segment	String	Product segment for categorization.
collection	String	Product collection grouping.
category	String	Product category (e.g., "Home Care").
product_type	String	Specific product type within category.
s_count_value_diy	Float	Quantity value (e.g., 500 for 500ml).
s_count_unit_diy	String	Quantity unit (e.g., "ml", "g", "pcs").
unit	String	Display unit for UI purposes.

2. Competitor Product

Fields describing the external product found via scraping.

Field Name	Type	Description
s_title_competitor	String	Competitor product name.
s_price_competitor	Float	Competitor price.
s_price_per_unit_competitor	Float	Competitor price per standardized unit.
s_primary_image_competitor	URL	Link to the competitor image.
s_vendor_competitor	String	Source platform (e.g., "Shopee", "Lazada").
s_link_competitor	URL	Link to the competitor listing.
s_count_value_competitor	Float	Competitor quantity value.
s_count_unit_competitor	String	Competitor quantity unit.
unit	String	Display unit for UI purposes.

3. Match Metrics

Scores indicating the quality of the relationship.

Field Name	Type	Description
matching_score	Float	Key Metric. A composite score (0-1) indicating overall confidence.
image_similarity	Float	Visual similarity score (0-1) based on embeddings.
exact_match	Number	Enhanced Classification. Numeric scale: 0 (broad), 1 (image-only), 2 (brand+image), 3 (exact).
brand_match	Number	Brand similarity flag (0 or 1).
scent_match	Number	Scent/fragrance similarity flag (0 or 1).